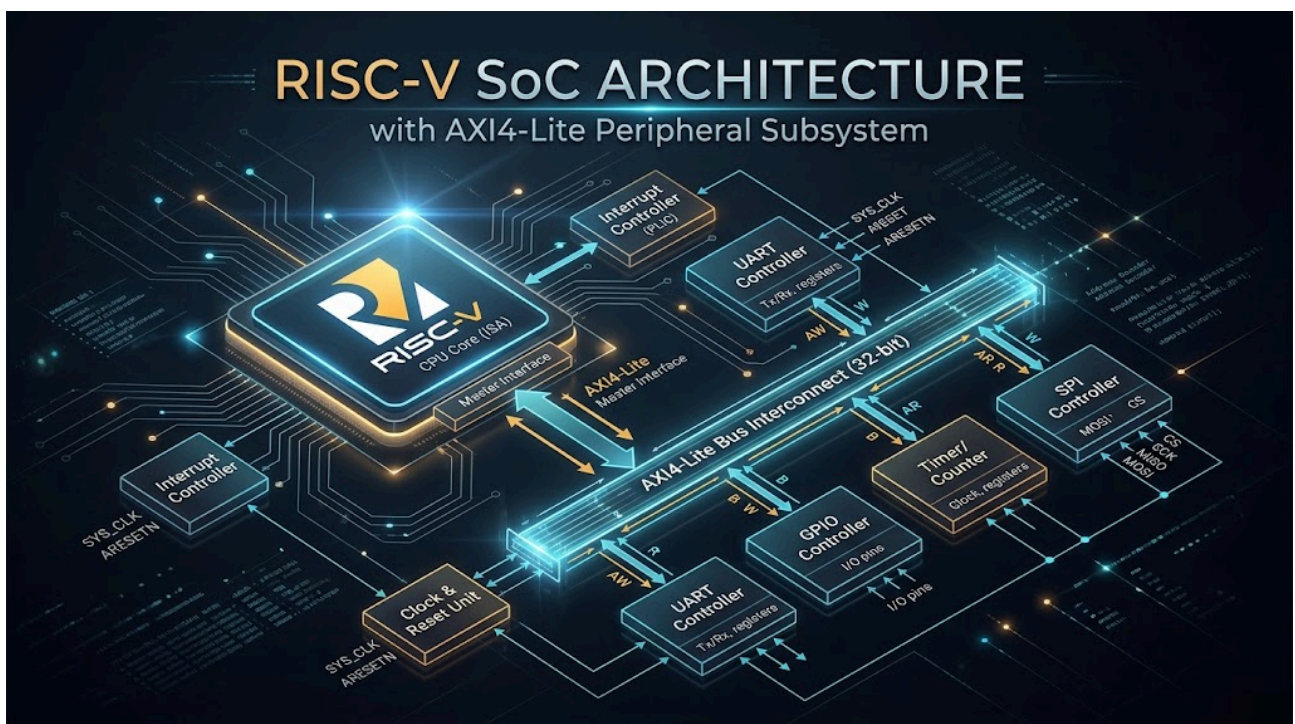


USTP CAPSTONE PROJECT • VERIFICATION TRACK

Technical Report



Team Members:

Muhammad Maaz Khan

Muhammad Saad

Danish Mehdi

Repository: <https://github.com/maaz-mk13/riscv-axi4lite-soc.git>

Table of Contents

1. Project Overview	2
2. System Architecture	3
2.1 Top-Level Block Diagram	3
2.2 RISC-V Core and Major Modules	3
2.3 Interfaces and Data/Control Flow	3
3. RTL Design	4
3.1 Module Hierarchy	4
	4
3.2 Description of Major RTL Modules	4
3.3 Important Design Decisions	6
3.4 Simulation Results/Waveforms	6
4. Verification	7
4.1 Verification Strategy	7
4.2 Testbench Architecture (Directed Layer)	7
4.3 Test Scenarios	7
4.4 Test Results	8
4.5 Assertions/Scoreboard	8
5. Track-Specific Work — UVM Verification	9
5.1 UVM Architecture	9
5.2 UVM Agent, Driver, Monitor	10
5.3 Sequencer/Sequences	10
5.4 Transaction	10
5.5 Environment	10
5.6 Scoreboard	10
5.7 Assertions — CPU-Bus SVA Checker	10
5.8 Functional Coverage	11
5.9 Code Coverage	11
6. Results	13
6.1 Simulation and Pass/Fail Results	13
6.2 Coverage Results	14
6.3 LEC / Tool Reports	14
6.4 Timing	14
7. Individual Contributions	15

1. Project Overview

Project Title: RISC-V SoC with AXI4-Lite Peripheral Subsystem

Project Objective: Design a synthesizable 32-bit single-cycle RV32I RISC-V core that communicates with memory-mapped peripherals through an AXI4-Lite subsystem, verify it with a self-checking directed testbench suite, and build a UVM verification environment covering the AXI4-Lite subsystem and all peripherals with functional coverage and code coverage.

Brief Description of the Proposed System: The system consists of a single-cycle RV32I core with separate instruction and data SRAM, a data bus decoder that routes the core's memory accesses to either data SRAM or an AXI4-Lite bus, a core-to-AXI master adapter, an AXI4-Lite interconnect with address decoding, and four memory-mapped peripherals (GPIO, Timer, PWM, UART) implemented as AXI4-Lite slaves. The core, memory, AXI subsystem, and peripherals were each independently verified with self-checking directed testbenches before integration; a UVM environment additionally drives the AXI subsystem and peripherals as a "virtual CPU" for randomized and directed verification, functional coverage, and SVA-based protocol checking.

2. System Architecture

2.1 Top-Level Block Diagram

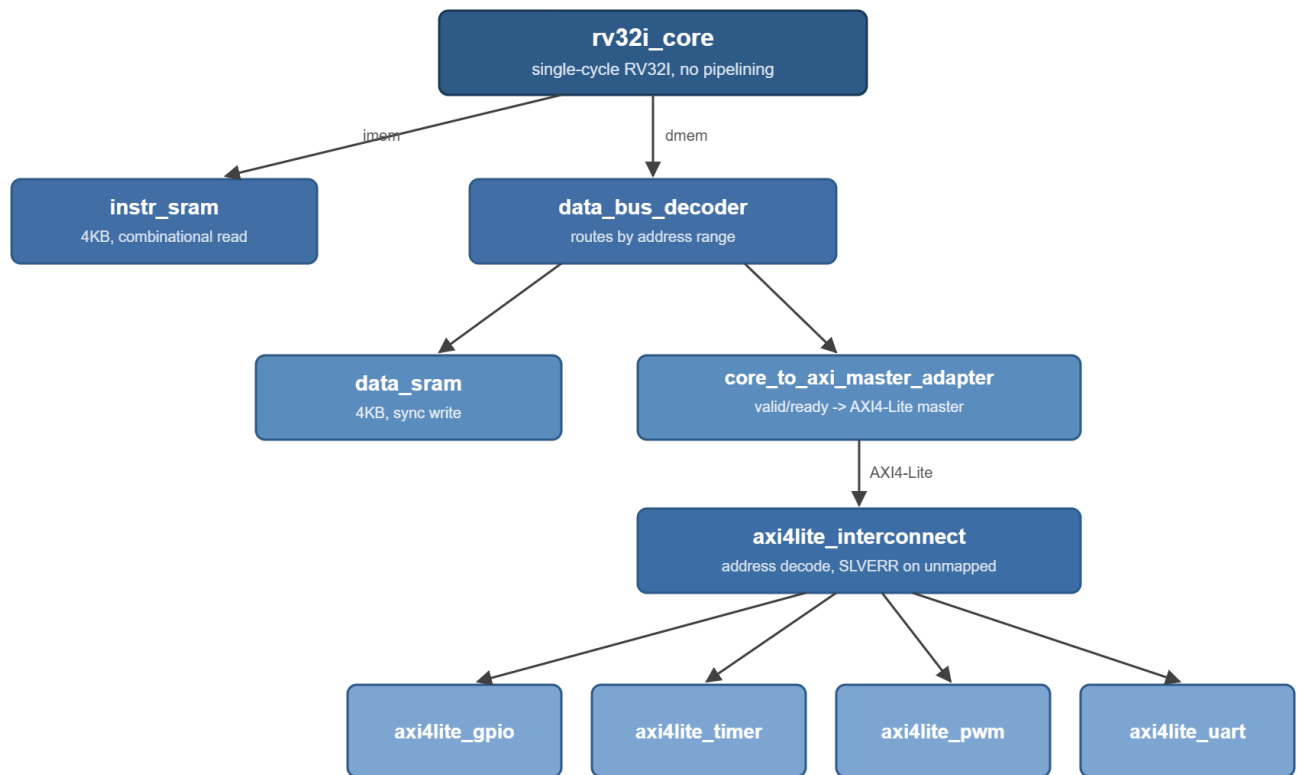


Figure 1 — RISC-V SoC with AXI4-Lite Peripheral Subsystem: Top-Level Block Diagram

2.2 RISC-V Core and Major Modules

rv32i_core is a single-cycle (non-pipelined) implementation of the RV32I base ISA, built from independently-testable submodules: **decode_unit** (instruction field/immediate extraction for all six RV32I formats), **control_unit** (control signal generation for every RV32I opcode category except ECALL/EBREAK/FENCE, which are treated as an explicit NOP), **regfile** (32×32-bit register file, x0 hardwired to zero), **alu** (10 RV32I ALU operations), and **load_store_unit** (byte/halfword store alignment and load sign/zero-extension).

2.3 Interfaces and Data/Control Flow

- **Core ↔ instruction memory:** combinational read — required for a genuinely single-cycle core (a registered SRAM would force multi-cycle fetch/execute).
- **Core ↔ data path:** a simple valid/ready/addr/wdata/wstrb/ rdata bus, identical in shape whether the target is data SRAM or an AXI peripheral. The core stalls (holds PC, withholds the register write) while `dmem_valid && !dmem_ready` — this is what

lets it tolerate a slower AXI peripheral without any core redesign.

- **data_bus_decoder**: combinational address-range routing between data_sram's window and the AXI path; an address in neither range gets an immediate response (never a hang).
- **AXI4-Lite**: the full 5-channel protocol (AW, W, B, AR, R) between core_to_axi_master_adapter and axi4lite_interconnect, and again between the interconnect and each peripheral slave port.

3. RTL Design

3.1 Module Hierarchy

```
soc_top
├── rv32i_core
│   ├── decode_unit
│   ├── control_unit
│   ├── regfile
│   ├── alu
│   └── load_store_unit
├── instr_sram
├── data_bus_decoder
├── data_sram
├── core_to_axi_master_adapter
├── axi4lite_interconnect
├── axi4lite_gpio
├── axi4lite_timer
├── axi4lite_pwm
└── axi4lite_uart
```

3.2 Description of Major RTL Modules

Module	Responsibility
decode_unit.sv	Extracts opcode/rd/rs1/rs2/funct3/funct7 and the

Module	Responsibility
	correctly sign-extended immediate for whichever RV32I format the opcode implies
control_unit.sv	Generates reg_write, alu_src, mem_read, mem_write, result_src, branch, jump, jalr, auipc, and alu_op for every RV32I opcode
regfile.sv	32×32-bit register file; x0 always reads zero regardless of any write attempt
alu.sv	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
load_store_unit.sv	SB/SH/SW byte-lane alignment; LB/LH/LW/LBU/LHU sign/zero-extension
rv32i_core.sv	Top-level integration: PC logic, branch/jump resolution, stall logic, writeback mux
instr_sram.sv / data_sram.sv	Combinational-read SRAM models, 4 KB each
data_bus_decoder.sv	Routes the core's data bus to data_sram or the AXI path by address
core_to_axi_master_adapter.sv	FSM converting the core's simple bus into AXI4-Lite master signals; asserts AWVALID/WVALID together (documented simplification for a single non-pipelined master)
axi4lite_interconnect.sv	Combinational address decode across 4 slave windows; generates SLVERR itself for any unmapped address so the core never hangs
axi4lite_gpio.sv	DATA_OUT/DATA_IN (2-flop synchronized)/DIR/STATUS registers
axi4lite_timer.sv	One-shot/periodic down-counter; RW1C expired status implemented as a one-cycle pulse on the actual zero-crossing (not a held level)

Module	Responsibility
axi4lite_pwm.sv	Free-running counter PWM; DUTY $\geq$ PERIOD and PERIOD=0 handled as explicit clamped edge cases
axi4lite_uart.sv	Fixed 8N1 framing, only the baud divisor is software-configurable; RX samples mid-bit for timing margin

3.3 Important Design Decisions

Decision	Rationale
Combinational (same-cycle) SRAM reads	Required for a true single-cycle, non-pipelined core
AWVALID/WVALID asserted together	Single non-pipelined master, no independent AW/W queuing
OKAY/SLVERR response codes only	Sufficient for this scope
ECALL/EBREAK/FENCE = explicit NOP	Documented, not silently dropped
UART fixed 8N1, baud divisor only configurable	Matches the "UART model" requirement without extra scope
rst used directly as a synchronous reset (no dedicated async-assert/sync-deassert synchronizer)	The originally proposed 2-flop reset synchronizer was not implemented in the final RTL; there is no genuinely asynchronous reset source in simulation
Sticky status flags (Timer expired, UART rx_valid) implemented as a one-cycle pulse on the actual event, never a held level	A held-level implementation was found (during testing) to fight software's RW1C/read-clear semantics; fixed and applied as a general rule across peripherals

3.4 Simulation Results/Waveforms

Representative passing waveforms were captured during development for: regfile write/readback, data_sram write timing, the GPIO write-readback path, the PWM 30% duty cycle, and the UART TX 8N1 frame for 0xA5.

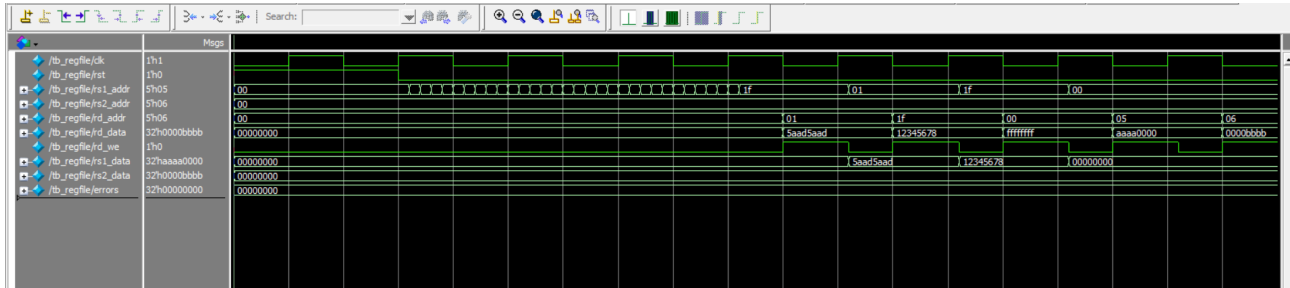


Fig 3.1: tb_regfile waveform: Writes commit exactly on the clock edge; x0 stays pinned at zero throughout

```
VSIM 133> run -all
# PASS full-chain write drove gpio_out = 0x5a
# PASS full-chain readback = 0x0000005a
# === TB_SOC_GPIO_INTEGRATION: ALL TESTS PASSED ===
# ** Note: $finish      : C:/questasim64_2024.1/examples/riscv-axi4lite-soc/tb_soc_gpio_integration.sv(150)
# Time: 100 ns Iteration: 1 Instance: /tb_soc_gpio_integration
# 1
```

Fig 3.2: Full SoC GPIO integration test transcript showing CPU-generated GPIO write/readback through the AXI4-Lite path and successful completion of all checks.

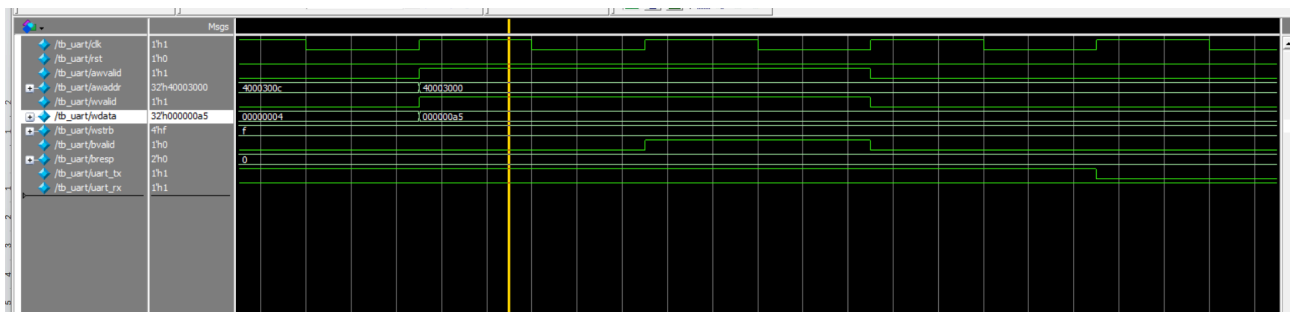


Fig 3.3: UART TX waveform for 0xA5, showing the 8N1 frame with start bit, eight LSB-first data bits, and stop bit.

4. Verification

4.1 Verification Strategy

Two complementary layers were used:

A. Directed/self-checking verification. Every RTL module has its own standalone, self-checking SystemVerilog testbench (no external scoreboard needed — each testbench computes its own expected values and reports PASS/FAIL directly). Modules were verified

bottom-up: register file → ALU → decode → control → full core → AXI

adapter/interconnect → each peripheral → full SoC integration → system-level

invalid-access/no-hang test.

B. UVM-based verification. A UVM environment (`axi_test_pkg`) drives the AXI4-Lite subsystem (adapter + interconnect + all 4 real peripherals) as a virtual CPU, using a shadow-register scoreboard, functional coverage, and directed + randomized sequences. This layer specifically targets address decoding, register read/write consistency across peripherals, and weighted-random exercising of all 4 peripheral windows plus the invalid address region.

4.2 Testbench Architecture (Directed Layer)

Each unit testbench instantiates its DUT directly and drives it with either hand-computed instruction encodings (`tb_core`, verified bit-by-bit against the RV32I encoding formulas — never assembled by an external tool) or direct AXI4-Lite slave-port stimulus (`tb_gpio`, `tb_timer`, `tb_pwm`, `tb_uart`). System-level tests (`tb_soc_top`, `tb_soc_gpio_integration`, `tb_soc_invalid_access`) instantiate `soc_top` and drive it only through the core's own instruction stream, proving the full chain rather than any individual module in isolation.

4.3 Test Scenarios

Layer	Scenarios covered
Core	All 10 ALU ops (including SLT/SLTU true <i>and</i> false comparison outcomes), all 6 instruction formats, taken-branch skip behavior, load/store byte addressing, a zero-result ALU case
AXI adapter + interconnect	Write/read round-trip to a slave, SLVERR generation and no-hang behavior for an unmapped address
GPIO	DATA_OUT/DIR write+readback, physical pin drive, DATA_IN via the input synchronizer, STATUS=0

Layer	Scenarios covered
Timer	Reset state, one-shot expiry, RW1C status clear, periodic wraparound (counter genuinely running, not frozen)
PWM	0%, 30%, 100% duty cycle (cycle-accurate), PERIOD=0 edge case, STATUS/pin consistency
UART	TX start/8 data bits/stop bit (byte-accurate 8N1 framing for 0xA5), RX frame reception (0x3C), rx_valid clear-on-read
System	Full-chain GPIO write/readback via real CPU instructions, no-hang proof on an unmapped address
UVM	Directed write-read per peripheral's key register, weighted-random address sequence across all 4 peripheral windows + the invalid region, a forced directed invalid-region access

4.4 Test Results

See §5 (Results) for the consolidated pass/fail table.

4.5 Assertions/Scoreboard

Scoreboard: axi_scoreboard (UVM) maintains a shadow-register model for every known RW register (GPIO DATA_OUT/DIR, Timer CTRL/LOAD, PWM CTRL/ PERIOD/DUTY, UART CTRL) and checks that a subsequent read of that address returns exactly what was last written. Live/read-only registers (TIMER_COUNT, *_STATUS, GPIO_DATA_IN, UART_RXDATA) are intentionally excluded from this shadow model — they are not deterministically shadow-predictable from the AXI interface alone, and are instead covered by the directed peripheral testbenches (§4.3).

Assertions: see §4.6 (Track-Specific Work) for full detail on axi_cpu_bus_checker.sv.

5. Track-Specific Work — UVM Verification

5.1 UVM Architecture

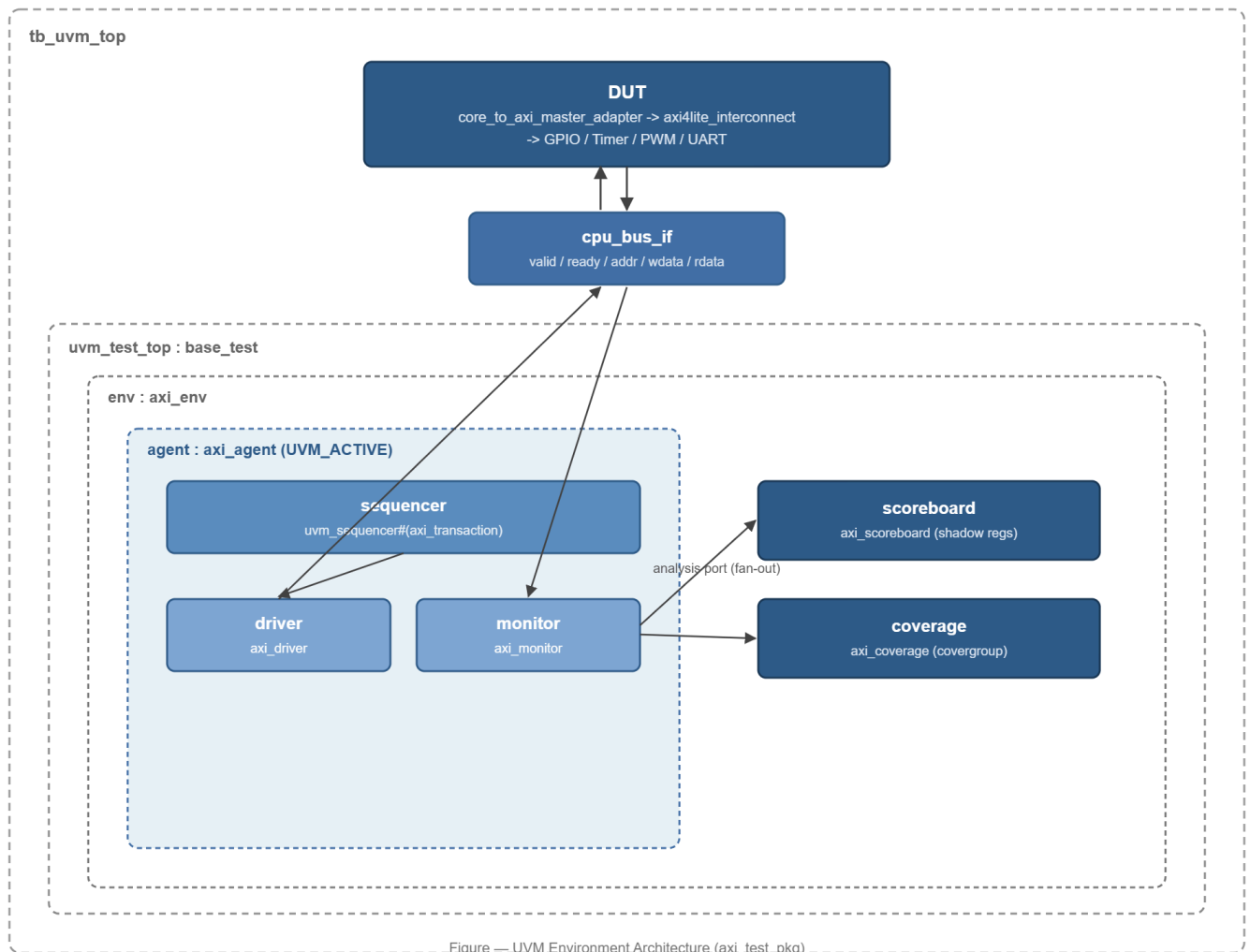


Figure — UVM Environment Architecture (axi_test_pkg)

```

tb_uvm_top
├── cpu_bus_if          (interface: the core's simple valid/
│                       ready bus, used as the UVM-driven
│                       interface into the real DUT)
├── DUT: core_to_axi_master_adapter
│   ├── axi4lite_interconnect
│   └── axi4lite_gpio / axi4lite_timer / axi4lite_pwm / axi4lite_uart
├── uvm_test_top : base_test
│   └── env : axi_env
│       ├── agent : axi_agent (UVM_ACTIVE)
│       │   ├── sequencer : uvm_sequencer#(axi_transaction)
│       │   ├── driver : axi_driver
│       │   └── monitor : axi_monitor
│       │       ├── analysis port ────> scoreboard : axi_scoreboard
│       │       └── coverage : axi_coverage
  
```

5.2 UVM Agent, Driver, Monitor

`axi_agent` is a standard active agent wrapping a driver, sequencer, and monitor. `axi_driver` drives `cpu_bus_if` exactly the way `rv32i_core` itself would — assert `valid/addr/wdata/wstrb`, wait for `ready`, capture `rdata` on a read. `axi_monitor` passively watches the same interface and publishes an `axi_transaction` object every cycle a transaction completes (`valid && ready`).

5.3 Sequencer/Sequences

`axi_write_read_seq` (directed) writes one register then reads it back. `axi_random_seq` generates `axi_transaction` objects with a weighted address distribution (`GPIO_BASE/TIMER_BASE/PWM_BASE/UART_BASE` windows plus a known-unmapped address). `base_test` runs one directed write-read per peripheral's key register, then a forced directed write-read to the invalid-address region (added specifically to close a functional-coverage bin that random weighting alone left empty), then the randomized sequence.

5.4 Transaction

`axi_transaction` (`uvm_sequence_item`) carries `addr`, `wdata`, `is_write`, and `rdata` (captured by the driver on a read). Its `addr_dist_c` constraint implements the weighted address distribution described above.

5.5 Environment

`axi_env` instantiates one `axi_agent` and connects its monitor's analysis port to both `axi_scoreboard` and `axi_coverage` (fan-out via `uvm_analysis_port`).

5.6 Scoreboard

Described in §4.5. Final result: **0 mismatches** across all runs.

```

# UVM_INFO axi_monitor.sv(33) @ 1636: uvm_test_top.env.agent.monitor [MONITOR] ADDR=40000007 WDATA=3304e3e2 WSTRB=0 IS_WRITE=0 RDATA=00000000
# UVM_INFO axi_monitor.sv(33) @ 1686: uvm_test_top.env.agent.monitor [MONITOR] ADDR=40002a9c WDATA=449f5cff WSTRB=0 IS_WRITE=0 RDATA=00000000
# UVM_INFO axi_monitor.sv(33) @ 1736: uvm_test_top.env.agent.monitor [MONITOR] ADDR=40002827 WDATA=4d2831a8 WSTRB=f IS_WRITE=1 RDATA=00000000
# UVM_INFO axi_monitor.sv(33) @ 1786: uvm_test_top.env.agent.monitor [MONITOR] ADDR=4000260e WDATA=e1b75f17 WSTRB=f IS_WRITE=1 RDATA=00000000
# UVM_INFO axi_monitor.sv(33) @ 1836: uvm_test_top.env.agent.monitor [MONITOR] ADDR=40000e2f WDATA=2978b4a7 WSTRB=0 IS_WRITE=0 RDATA=00000000
# UVM_INFO axi_monitor.sv(33) @ 1886: uvm_test_top.env.agent.monitor [MONITOR] ADDR=400025dc WDATA=6afc273f WSTRB=0 IS_WRITE=0 RDATA=00000000
# UVM_INFO axi_monitor.sv(33) @ 1936: uvm_test_top.env.agent.monitor [MONITOR] ADDR=400014d3 WDATA=01ef18c6 WSTRB=0 IS_WRITE=0 RDATA=00000000
# UVM_INFO axi_monitor.sv(33) @ 1986: uvm_test_top.env.agent.monitor [MONITOR] ADDR=40003593 WDATA=ee583812 WSTRB=0 IS_WRITE=0 RDATA=00000000
# UVM_INFO verilog_src/uvvm-1.1d/src/base/uvvm_objection.svh(1267) @ 2005: reporter [TEST DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO axi_coverage.sv(50) @ 2005: uvm_test_top.env.coverage [COVERAGE] Address-region x direction cross coverage = 100.00%
# UVM_INFO axi_scoreboard.sv(67) @ 2005: uvm_test_top.env.scoreboard [SCOREBOARD] Checked 4 shadow-predictable reads, 0 mismatch(es)
# UVM_INFO axi_scoreboard.sv(71) @ 2005: uvm_test_top.env.scoreboard [SCOREBOARD] === SCOREBOARD: ALL CHECKS PASSED ===

# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 47
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
#
# ** Report counts by id
# [COVERAGE] 1
# [MONITOR] 40
# [Questa UVM] 2
# [RNTST] 1
# [SCOREBOARD] 2
# [TEST_DONE] 1
#
# ** Note: $finish : C:/questasim64_2024.1/win64/./verilog_src/uvvm-1.1d/src/base/uvvm_root.svh(430)
# Time: 2005 ns Iteration: 61 Instance: /tb_uvvm_top

```

Fig 5.1: UVM verification transcript showing successful scoreboard checks, zero UVM errors/fatal errors, and 100% functional coverage.

5.7 Assertions — CPU-Bus SVA Checker

axi_cpu_bus_checker.sv is bound automatically onto every instance of cpu_bus_if via axi_bind.sv (bind cpu_bus_if axi_cpu_bus_checker ...), so it observes the interface without needing manual instantiation in each testbench. It checks, independently of any specific directed test:

1. **valid is never X/Z.**
2. **addr remains stable** while a transaction is pending (valid && !ready).
3. **wdata/wstrb remain stable** while a write is pending.
4. **ready eventually arrives** within a 200-cycle bound after valid asserts — a direct, general liveness/no-hang check independent of any specific directed hang test.

No assertion failures were observed in any tb_uvvm_top run — confirmed by the absence of any PROTOCOL:-prefixed error in the simulation transcripts across all runs performed.

5.8 Functional Coverage

axi_coverage (UVM uvm_subscriber) implements a covergroup crossing address region (gpio/timer/pwm/uart/invalid) against transaction direction (write/read) — $5 \times 2 = 10$ cross bins.

Result: 17/17 covergroup bins = 100.00% (the coverpoints themselves — 5 region bins + 2 direction bins — plus the 10-bin cross, all covered). This was reached after adding the

directed invalid-region sequence in §5.3; pure random weighting alone left the invalid-region bins empty in an earlier run, which is why that directed sequence exists.

5.9 Code Coverage

Code coverage (-cover sbceft: statement, branch, condition, expression, FSM, toggle) was collected per-testbench and merged with vcover merge across every directed and UVM testbench.

Final merged result (Instance view, filtered): 71.61% total.

Metric	Coverage (Hits/Bins)
Branches	73.03% (650/890)
Conditions	40.00% (76/190)
Covergroups	100.00% (1/1)
Expressions	58.74% (84/143)
FSM States	90.32% (56/62)
FSM Transitions	73.68% (56/76)
Statements	87.70% (1819/2074)
Toggles	49.46% (5855/11836)
Total coverage (filtered view)	71.61%

Per-testbench code coverage from the latest merged run (each row is the aggregate coverage report Questa printed right after that testbench's own run, not an isolated DUT-only figure — see note below):

Testbench (aggregate report)	Branches	Conditions	Statements	Toggles
alu — tb_alu	84.21%	50.00%	86.11%	70.05%
control_unit — tb_control	90.90%	0.00%	98.09%	52.53%
regfile — tb_regfile	76.92%	50.00%	90.00%	49.76%

Testbench (aggregate report)	Branches	Conditions	Statements	Toggles
core_to_axi_master_adapter — tb_adapter_coverage	100.00%	75.00%	98.85%	99.62%
axi4lite_uart — tb_uart	75.78%	52.00%	90.74%	26.66%
axi4lite_pwm — tb_pwm	83.56%	46.15%	89.71%	59.74%

Note on interpretation: this table reports the aggregate coverage Questa printed for each standalone testbench compile (coverage report -summary immediately after that testbench's run), not an isolated DUT-only instance figure — each total therefore includes the testbench wrapper and any shared package alongside the module under test. These aggregate numbers are the most current data available from the latest coverage run; a DUT-only instance breakdown would need a separate per-instance report.

Toggle coverage is the dominant remaining gap and is largely architectural, not a testing omission: several registers are 32-bit buses where only a handful of low bits are ever meaningful (GPIO is 8 bits wide, register offsets are 4 bits, AXI response codes are 2 bits) — the unused upper bits cannot toggle under any realistic stimulus without violating the AXI4-Lite address map or inventing out-of-spec test conditions. This was verified against the coverage report's specific missed-toggle bin lists (e.g. awaddr[13:29], BRESP[0:1], bresp[0:1]) before being accepted as a documented limitation rather than pursued further.

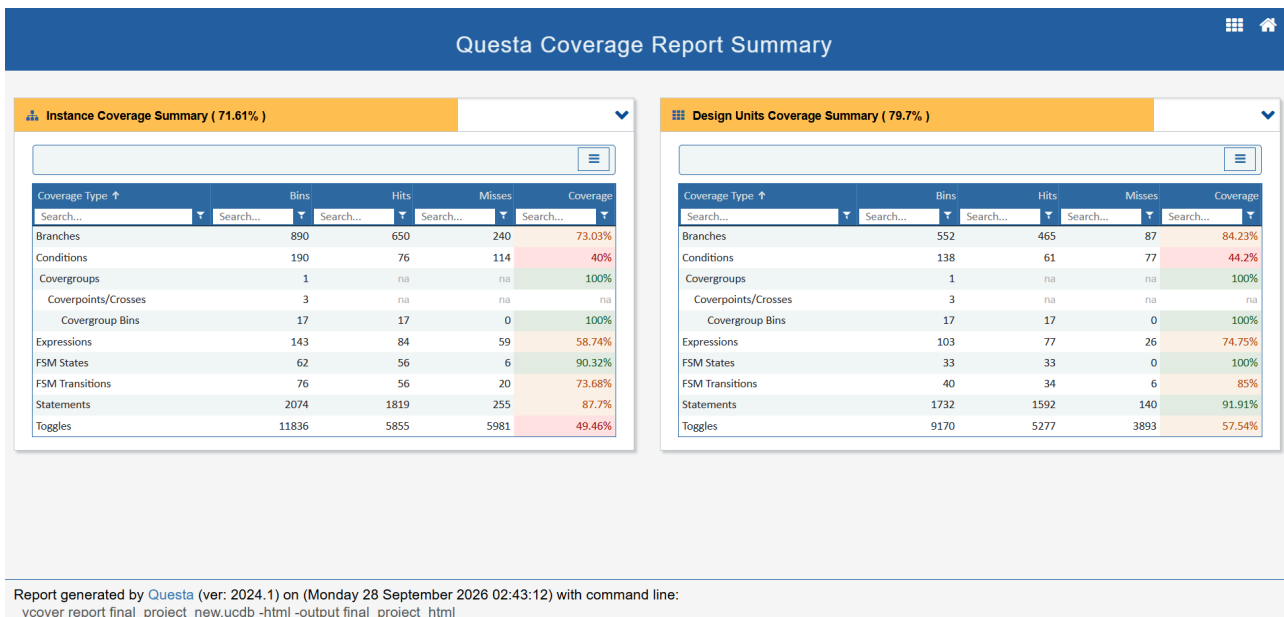
6. Results

6.1 Simulation and Pass/Fail Results

Testbench	Scope	Result
tb_regfile	Register file	ALL TESTS PASSED
tb_alu	ALU (10 ops)	ALL TESTS PASSED
tb_decode	Instruction decode (6 formats)	ALL TESTS PASSED
tb_control	Control unit (11 directed cases)	ALL TESTS PASSED
tb_core	Full core, 21-instruction program	ALL TESTS PASSED
tb_axi_path	AXI adapter + interconnect + dummy slave	ALL TESTS PASSED
tb_gpio	GPIO peripheral	ALL TESTS PASSED
tb_timer	Timer peripheral	ALL TESTS PASSED
tb_pwm	PWM peripheral	ALL TESTS PASSED
tb_uart	UART peripheral	ALL TESTS PASSED
tb_soc_gpio_integration	GPIO through the full chain	ALL TESTS PASSED
tb_soc_top	Full SoC, real CPU instructions	ALL TESTS PASSED
tb_soc_invalid_access	System-level no-hang proof	ALL TESTS PASSED (no hang confirmed)
tb_uvm_top (base_test)	UVM environment	UVM_ERROR=0, UVM_WARNING=0, UVM_FATAL=0, 0 scoreboard mismatches

6.2 Coverage Results

Metric	Result
Functional coverage (UVM covergroup)	100.00% (17/17 bins)
Code coverage — final merged total (Instance view, filtered)	71.61%
Code coverage — standalone-instance highlights	See §5.9 table



6.3 LEC / Tool Reports

Not applicable to this track. Logical Equivalence Checking is a Physical Design-track deliverable per the evaluation announcement; this project is Verification track. No LEC was performed and none is claimed.

6.4 Timing

An SDC constraints file (soc_top.sdc) was written, specifying a 100 MHz (10.000 ns) primary clock, 0.500 ns clock uncertainty, and 2.000 ns input/ output delay constraints on all external ports. **No synthesis or static timing analysis was run against this SDC** —

no timing closure is claimed. The SDC file explicitly documents that no asynchronous-reset synchronizer is implemented in the current RTL (rst is used directly as a synchronous reset throughout), which is why reset is constrained as an ordinary synchronous input rather than a false path.

7. Individual Contributions

Muhammad Maaz Khan — RV32I Core & Memory: regfile.sv, alu.sv, decode_unit.sv, control_unit.sv, load_store_unit.sv, rv32i_core.sv, instr_sram.sv, data_sram.sv, data_bus_decoder.sv, and their unit testbenches (tb_regfile, tb_alu, tb_decode, tb_control, tb_core) and test programs.

Muhammad Saad — AXI4-Lite Subsystem & UVM: core_to_axi_master_adapter.sv, axi4lite_interconnect.sv, tb_axi_path.sv, the full UVM environment (axi_transaction, axi_driver, axi_monitor, axi_scoreboard, axi_coverage, axi_agent, axi_env, axi_sequences, base_test), the SVA checker (axi_cpu_bus_checker.sv, axi_bind.sv), and code coverage collection/merging.

Danish Mehdi — Peripherals & System Integration: axi4lite_gpio.sv, axi4lite_timer.sv, axi4lite_pwm.sv, axi4lite_uart.sv and their standalone testbenches, soc_top.sv, tb_soc_top.sv, tb_soc_gpio_integration.sv, tb_soc_invalid_access.sv, soc_top.sdc, and this documentation set.

Joint work (all members): technical report, presentation, and GitHub repository maintenance/updates were done collaboratively by all three team members.